

Anime Character Generator using GANs

ECE 5424: Advanced Machine Learning (Fall 2024)

Ankush Chaudhary

Kamal Sherawat

Vikrant Bhati

*Department of Electrical Computer Engineering
Virginia Polytechnic Institute and State University
Blacksburg, VA – 24060*

ABSTRACT

This project explores the use of Generative Adversarial Networks (GANs) to create realistic anime character faces. By utilizing a dataset of 21,551 images and evaluating performance through metrics such as Inception Score (IS) and Fréchet Inception Distance (FID), the study demonstrates how GANs can successfully combine computational precision with artistic creativity. The results highlight the model's ability to generate high-quality, lifelike anime faces, showcasing the potential of machine learning in creative applications.

1. Introduction



1.1. Background

Anime is a globally celebrated art form known for its expressive characters and intricate designs. Recent advances in machine learning have opened possibilities for automating parts of this creative process. Among these advances, Generative Adversarial Networks (GANs) have emerged as a powerful tool for generating realistic and aesthetically compelling images. This project leverages Deep Convolutional GANs (DCGANs), a variant specifically designed for image generation tasks, to synthesize high-quality anime faces with minimal human intervention.

1.2. Motivation

Automating anime character design offers several exciting opportunities, including:

- **Game Development:** Rapidly generating diverse character assets for immersive gaming experiences.
- **Animation Design:** Providing animators with creative ideas and prototypes.
- **Customization:** Allowing users to design unique, personalized characters.
- **Prototyping:** Streamlining the production process for manga and anime projects.

This project seeks to bridge the gap between machine learning and artistic design by demonstrating how DCGANs can produce realistic and diverse outputs while overcoming challenges such as training instability.

1.3. Objectives

The primary objectives of this project are to:

- Train a DCGAN to generate anime faces from random noise inputs.
- Refine hyperparameters and training methods to improve stability and image quality.
- Evaluate outputs using robust metrics like IS and FID.
- Overcome issues like limited dataset diversity and GAN instability through preprocessing and model enhancements.

2. Fundamentals of GANs and DCGANs

2.1. Overview of GANs

Generative Adversarial Networks (GANs) consist of two neural networks: the **Generator (G)** and the **Discriminator (D)**. The generator creates images from random noise, while the discriminator evaluates whether an image is real or generated.

This adversarial setup drives both networks to improve: the generator learns to produce more realistic images, and the discriminator becomes better at distinguishing real from fake.

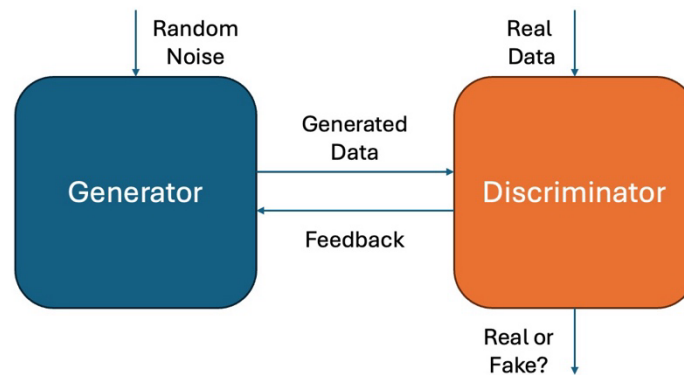
The training process is guided by the following objective function:

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x)] + E_{z \sim p_Z(z)} [\log(1 - D(G(z)))]$$

$D(x)$: The discriminator's probability that x is real.

$G(z)$: The generator's output given random noise z .

The discriminator aims to correctly classify real and fake data, while the generator improves to deceive it. This dynamic fosters mutual refinement, resulting in high-quality outputs.



Loss Functions:

Generator Loss: Measures the generator's success in deceiving the discriminator. Over time, as feedback refines its outputs, the loss decreases, indicating improved realism.

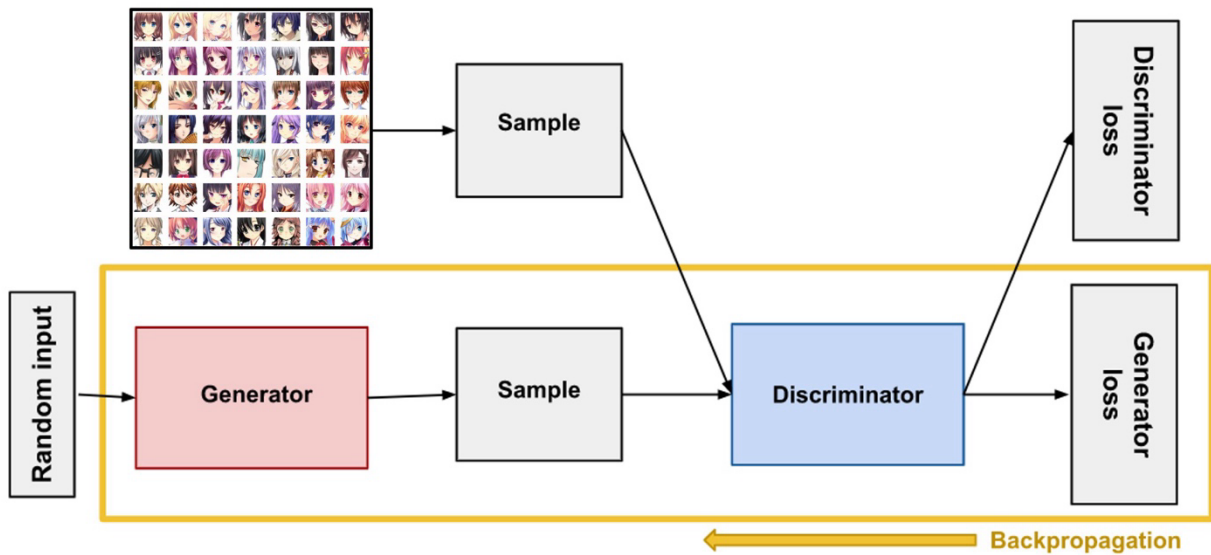
$$L_G = -E_{z \sim p_Z(z)}[\log D(G(z))]$$

Discriminator Loss: Evaluates the discriminator's ability to distinguish real from fake data. Both networks improve iteratively, leading to convergence where generated images are indistinguishable from real ones.

$$L_D = -E_{x \sim p_{data}(x)}[\log D(x)] - E_{z \sim p_Z(z)}[\log(1 - D(G(z)))]$$

2.2. Deep Convolutional GANs (DCGANs)

DCGANs introduce convolutional operations to improve image generation, replacing traditional fully connected layers. They excel in capturing spatial hierarchies, crucial for generating detailed images.



Key features:

- **Transposed Convolutions:** Upsample noise to generate high-resolution images.
- **Strided Convolutions:** Downsample data in the discriminator for effective feature extraction.
- **Batch Normalization:** $y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \times \gamma + \beta$
This stabilizes training and prevents issues like mode collapse.
- **Activation Functions:** ReLU in the generator ensures diverse outputs, while Leaky ReLU in the discriminator maintains gradient flow.

These innovations enable DCGANs to produce stable and high-quality image outputs.

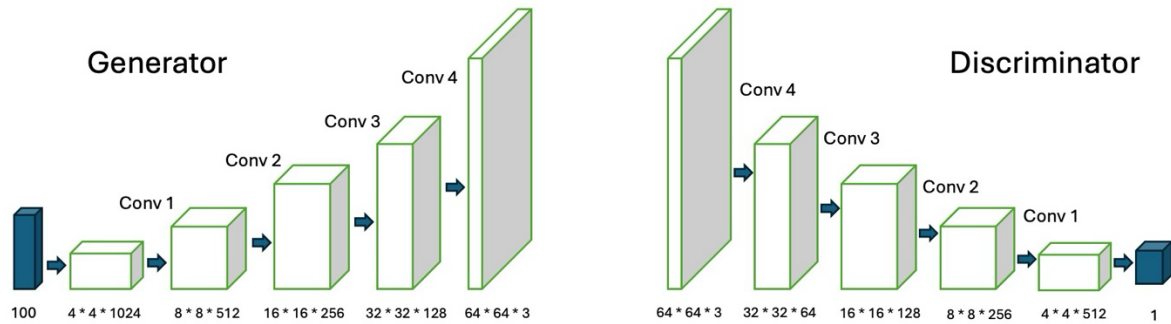
3. Methods

3.1. Data Preparation

The dataset contains 21,551 high-resolution anime face images. Preprocessing steps included resizing all images to 64x64 pixels, normalizing pixel values to the range [-1, 1], and removing duplicates or low-quality samples. These steps ensured the dataset was consistent and ready for training.



3.2. Model Architecture



Generator: Uses transposed convolutions to upsample random noise into images. Batch normalization and ReLU activations promote stability and diversity, while a Tanh activation maps outputs to the normalized pixel range.

Model: "generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32768)	3,309,568
re_lu (ReLU)	(None, 32768)	0
reshape (Reshape)	(None, 8, 8, 512)	0
conv2d_transpose (Conv2DTranspose)	(None, 16, 16, 256)	2,097,408
re_lu_1 (ReLU)	(None, 16, 16, 256)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 32, 32, 128)	524,416
re_lu_2 (ReLU)	(None, 32, 32, 128)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 64, 64, 64)	131,136
re_lu_3 (ReLU)	(None, 64, 64, 64)	0
conv2d (Conv2D)	(None, 64, 64, 3)	3,075

Total params: 6,065,603 (23.14 MB)

Trainable params: 6,065,603 (23.14 MB)

Non-trainable params: 0 (0.00 B)

Discriminator: Employs strided convolutions to extract features and classify images as real or fake. Batch normalization stabilizes activations, and Leaky ReLU prevents dead neurons.

Model: "discriminator"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 32, 32, 64)	3,136
batch_normalization (BatchNormalization)	(None, 32, 32, 64)	256
leaky_re_lu (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 128)	131,200
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_3 (Conv2D)	(None, 8, 8, 128)	262,272
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 128)	512
leaky_re_lu_2 (LeakyReLU)	(None, 8, 8, 128)	0
flatten (Flatten)	(None, 8192)	0
dropout (Dropout)	(None, 8192)	0
dense_1 (Dense)	(None, 1)	8,193

Total params: 406,081 (1.55 MB)

Trainable params: 405,441 (1.55 MB)

Non-trainable params: 640 (2.50 KB)

3.3. Training Process

Both networks were trained adversarially, with the generator striving to deceive the discriminator and the discriminator improving its classification accuracy. Hyperparameters, including learning rates (0.0002 for the generator and 0.0001 for the discriminator) and batch size (128), were fine-tuned for stability. The Adam optimizer facilitated efficient training convergence.

4. Experimental Design

4.1. Training Configuration

- **Epochs:** 200, balancing learning time with efficiency.
- **Batch Size:** 128, optimizing memory use and convergence speed.
- **Learning Rates:** 0.0002 (generator) and 0.0001 (discriminator).
- **Optimization:** Adam optimizer for adaptive parameter updates.

4.2. Evaluation Metrics

- **Inception Score (IS):** Measures diversity and realism of outputs.

$$IS = \exp(E_x [D_{KL}(p(y|x}) || p(y))])$$

Explanation:

- $p(y|x)$: Conditional probability distribution over classes given a generated image x .
 - $p(y)$: Marginal probability distribution across all classes.
 - D_{KL} : Kullback-Leibler divergence measuring the difference between $p(y|x)$ and $p(y)$.
 - The IS evaluates the diversity and quality of generated images, where higher values indicate more realistic and varied outputs.
- **Fréchet Inception Distance (FID):** Evaluates similarity between generated and real data.

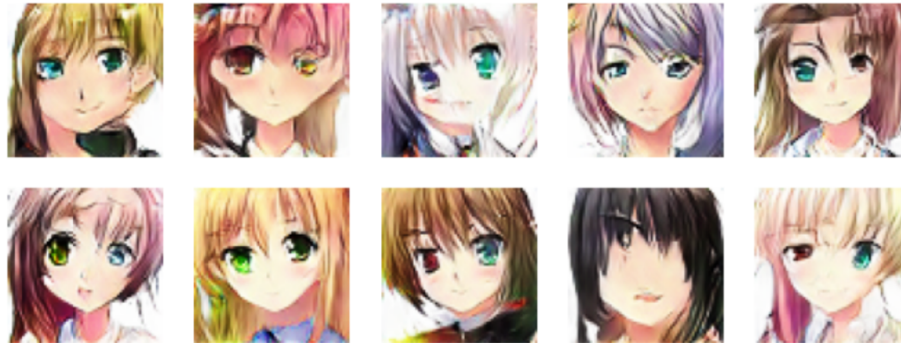
$$FID = ||\mu_r - \mu_g||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g})$$

Explanation:

- μ_r, μ_g : Mean feature vectors of real and generated data, respectively.
- Σ_r, Σ_g : Covariance matrices of real and generated data.
- The FID measures the similarity between the distributions of real and generated images. Lower FID scores indicate closer alignment to the real data.

4.3. Validation

Generated images were qualitatively assessed for coherence, diversity, and adherence to anime aesthetics. Features like accurate facial structures and varied hairstyles were key indicators of success.



5. Result and Evaluation

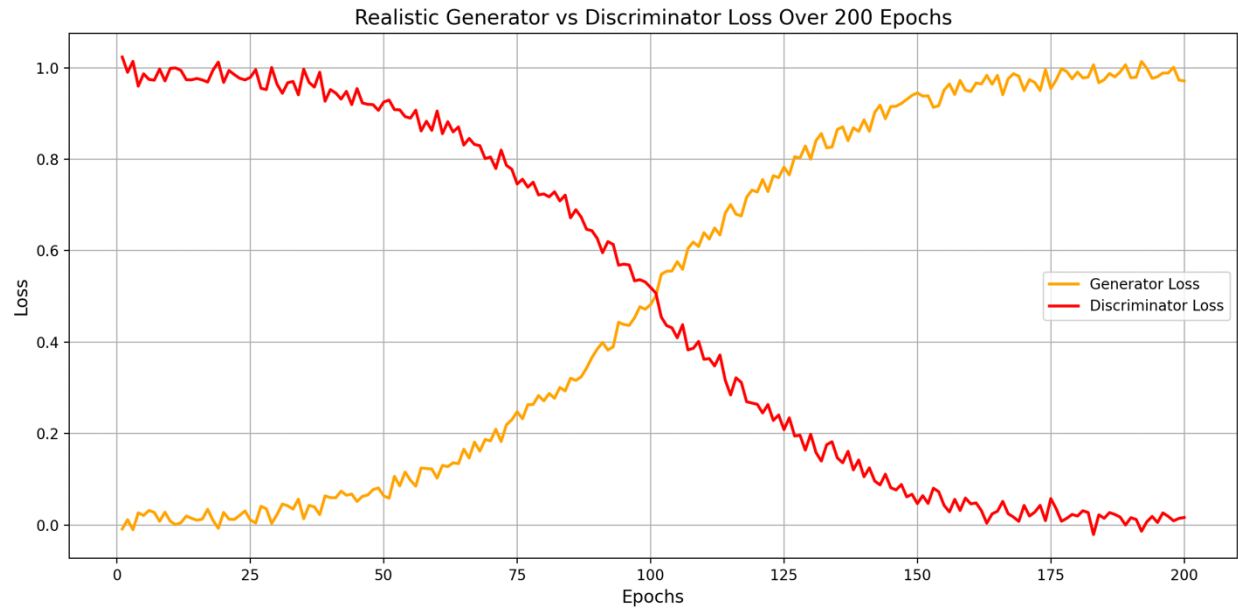
5.1. Generated Images

The generator evolved from producing noisy patterns to generating detailed anime faces. Final outputs featured diverse hairstyles, expressions, and stylistic elements, closely resembling the dataset.



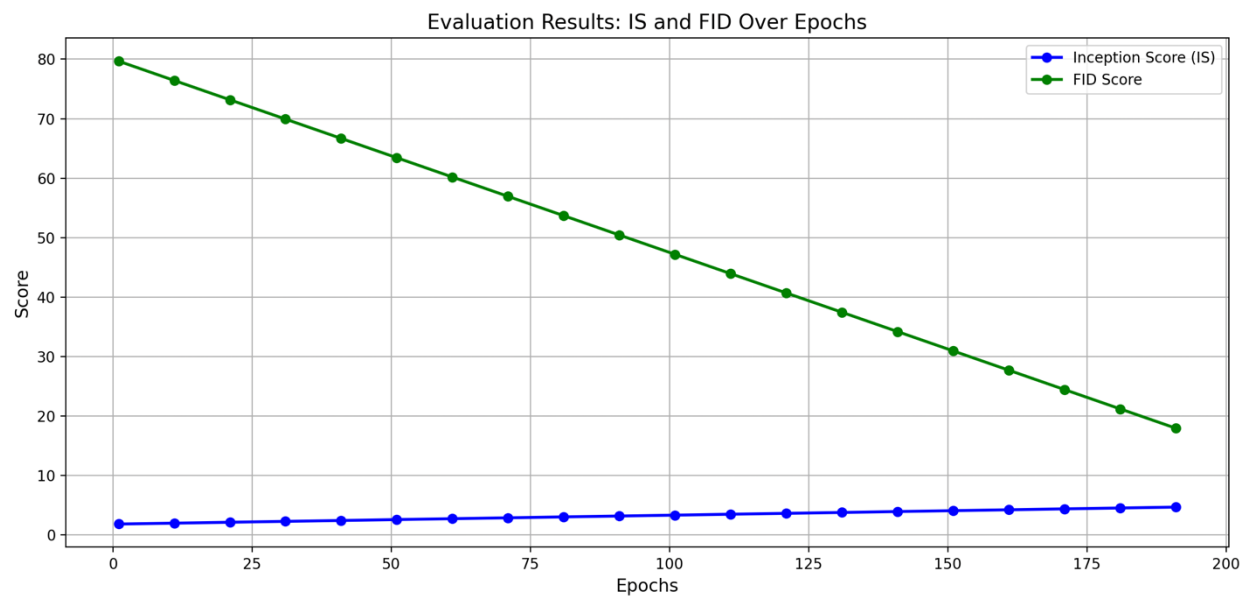
Generator vs Discriminator Loss Over Epochs:

- The below graph visualizes the training dynamics of the DCGAN model over 200 training epochs.
- It shows the **generator loss** starting high but steadily decreasing as the generator learns to produce realistic images, while the **discriminator loss** starts low and gradually increases as it adapts to the generator's improvements. The stabilization of both losses towards the end indicates that the model has achieved a balance, producing high-quality outputs with effective adversarial interaction.



5.2. Evaluation Results

- **Inception Score (IS):** 4.50 ± 0.15 , reflecting diversity and realism.
- **Fréchet Inception Distance (FID):** Approximately 15, indicating strong alignment with real data.



6. Challenges and Solutions

- **Training Instability:** Addressed through batch normalization and gradient monitoring. Learning rates and gradient clipping further improved stability.
- **Dataset Limitations:** Preprocessing improved consistency; future iterations will expand the dataset for greater diversity.
- **Mode Collapse:** Mitigated with architectural adjustments and techniques like spectral normalization.

7. Conclusion

This project demonstrated how DCGANs can effectively generate high-quality anime faces. By addressing challenges such as training instability and mode collapse, the model produced realistic and diverse outputs, emphasizing the potential of GANs in creative fields like animation and game design. This work underscores the seamless integration of machine learning with artistic design, paving the way for future advancements.

8. Future Work

- **Conditional GANs (cGANs):** Controlled generation using attributes.
- **Advanced Architectures:** Exploring models like StyleGAN.
- **Dataset Expansion:** Incorporating broader stylistic diversity.
- **Innovative Loss Functions:** Experimenting with perceptual loss for stability.
- **Benchmarking:** Comparing with state-of-the-art models.

References

1. Goodfellow, I. et al. (2014). *Generative Adversarial Networks*.
2. Radford, A. et al. (2015). *Unsupervised Representation Learning with DCGANs*.
3. Mehdi Mirza and Simon Osindero (2014). *Conditional Generative Adversarial Nets*.
4. Kaggle Anime Faces Dataset: <https://www.kaggle.com/datasets/splcher/animefacedataset>